Name: Casey Tom | **Student ID:** 1230794853

Project 1: Density Estimation and Classification Using Naive Bayes

Project Analysis and Objective

The objective of this project was to implement a Gaussian Naive Bayes classifier for handwritten MNIST images of digits 0 and 1. The implementation involved extracting the required features, estimating Gaussian parameters from the training data, implementing the probability calculations directly, and validating the completed classifier using the Ed Lessons autograder. Each MNIST image contained $28 \times 28 = 784$ pixel values. The project reduced each image to two features: Feature 1, the average pixel brightness, and Feature 2, the standard deviation of the pixel values. The training data contained 5,000 images of each digit, while the test sets contained 980 digit-0 images and 1,135 digit-1 images. The class priors were specified as $P(Y=0) = P(Y=1) = 0.5$.

Solution Approach

First, NumPy was used to extract two features from each 28×28 image: the mean pixel value and the standard deviation of the pixel values. The same feature-extraction procedure was applied to both the training and test sets. Second, only the training features were used to estimate four Gaussian distributions, one for each feature-class combination. Each distribution required a mean and a variance, resulting in eight parameters in total, which were calculated using `np.mean()` and `np.var()`. An important distinction is that Feature 2 represents the standard deviation of the pixels within one image, while its Gaussian parameter is the variance of the Feature-2 values across an entire training class. Third, the Gaussian probability density function was implemented manually rather than using a packaged Naive Bayes classifier or library Gaussian PDF function: $p(x \mid \mu, \sigma^2) = 1 / \text{sqrt}(2\pi\sigma^2) \times \exp(-(x - \mu)^2 / (2\sigma^2))$. For each test image,

the following Naive Bayes scores were calculated: score0 = P(Y=0) × p(feature1 | Y=0) × p(feature2 | Y=0) and score1 = P(Y=1) × p(feature1 | Y=1) × p(feature2 | Y=1). The classifier assigned the digit corresponding to the larger score. This calculation applies the Naive Bayes conditional-independence assumption by multiplying the two feature likelihoods. Finally, accuracy was calculated separately for each test set as the proportion of correctly classified images. The final accuracies were approximately 0.91735 for digit 0 and 0.92335 for digit 1.

Difficulties, Verification, and Lessons Learned

The main implementation challenges involved working with the starter notebook structure and keeping the roles of standard deviation and variance clear. Because the starter main() function loaded the datasets as local variables, the generated .mat files were loaded in a separate cell for the later tasks. Care was also required to distinguish Feature 2's per-image standard deviation from the variance estimated across each training class and to ensure that digit-0 and digit-1 likelihoods used the correct class-specific Gaussian parameters. The project was validated incrementally by checking the dataset dimensions, extracted feature arrays, Gaussian parameters, prediction counts, and final accuracies as each task was completed. After implementation, the kernel was restarted and Run All was used to confirm that the complete notebook executed successfully without relying on previously stored variables. The completed notebook was also verified in Ed Lessons, where all six autograder test cases passed. The project provided a clearer understanding of how extracted image features are used to estimate Gaussian distributions and how those distributions are combined within a Naive Bayes classifier. It also reinforced the importance of estimating model parameters only from training data, checking intermediate results, implementing mathematical formulas carefully, and verifying the complete classifier before relying on its final output.